

Placement Pipeline

Visualization Report

NestingRL — Placement Network

May 18, 2026

Checkpoint: `checkpoints/perthet_combined/final.pt`

12 convex val pairs (`orig_idx` $\in [10800, 12000)$), 6 concave-fs val pairs (`bo_train_pool_10k` position $\in [9000, 10000)$).

Refinement window: $\pm 10 \text{ px} \times \pm 2 \theta\text{-bins} \Rightarrow 2205$ Shapely calls per pair.

Contents

1	Placement Network — Methodology	2
1.1	Formal problem statement	2
1.1.1	Notation	2
1.1.2	Validity and reward	2
1.1.3	Optimization objective	3
1.1.4	Quality metric	4
1.2	Model	4
1.2.1	Inputs	4
1.2.2	Architecture: π_ϕ	4
1.2.3	Why this conditioning works	5
1.3	Training objective	5
1.3.1	Target distribution	5
1.3.2	Loss	6
1.3.3	Training-step distribution	6
1.3.4	Augmentation: dihedral group D_2	6
1.3.5	Optimization	6
1.4	Inference	6
1.5	Data pipeline	7
1.5.1	Pair generation	7
1.5.2	IFP precompute	7
1.5.3	Reward heatmap precompute	7
1.5.4	Rotated-part-mask precompute	7
1.5.5	Combined corpus	7

1.6	Results	7
1.7	Implementation pointers	8
2	Examples — Convex–convex pairs	9
2.1	Pair convex_10819 (area ratio 0.214)	9
2.2	Pair convex_10848 (area ratio 0.425)	10
2.3	Pair convex_10889 (area ratio 0.468)	11
2.4	Pair convex_11009 (area ratio 0.140)	12
2.5	Pair convex_11121 (area ratio 0.059)	13
2.6	Pair convex_11167 (area ratio 0.079)	14
2.7	Pair convex_11408 (area ratio 0.542)	15
2.8	Pair convex_11557 (area ratio 0.338)	16
2.9	Pair convex_11578 (area ratio 0.575)	17
2.10	Pair convex_11774 (area ratio 0.390)	18
2.11	Pair convex_11811 (area ratio 0.519)	19
2.12	Pair convex_11895 (area ratio 0.131)	20
3	Examples — Concave free-space pairs	21
3.1	Pair concave_09002 (area ratio nan)	21
3.2	Pair concave_09033 (area ratio nan)	22
3.3	Pair concave_09392 (area ratio nan)	23
3.4	Pair concave_09553 (area ratio nan)	24
3.5	Pair concave_09764 (area ratio nan)	25
3.6	Pair concave_09854 (area ratio nan)	26
4	Summary	27

1 Placement Network — Methodology

A learned placement function that, given a free-space polygon F and a part polygon P , returns a placement (θ, x, y) maximizing a Shapely-defined reward. Trained on 22 000 pairs (12 k convex–convex + 10 k concave–fs–convex), achieves val recovery ≈ 0.72 with one small U-Net and one feed-forward pass per rotation.

1.1 Formal problem statement

1.1.1 Notation

Let

- $F \subset \mathbb{R}^2$ be a simple polygon (the *free space*), possibly non-convex.
- $P \subset \mathbb{R}^2$ be a simple convex polygon (the *part*), centered at the origin so that its centroid coincides with $(0, 0)$.
- $R_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ denote rotation by angle θ about the origin.
- $T_{(x,y)}$ denote translation by (x, y) .
- The *placement* of P at position (x, y) with rotation θ is the set

$$P_{\theta,x,y} = T_{(x,y)}(R_\theta(P)).$$

We discretize:

- Rotations: $\theta \in \Theta = \{0, \frac{2\pi}{36}, \frac{4\pi}{36}, \dots, \frac{70\pi}{36}\}$, indexed $i = 0, \dots, 35$.
- Positions: $(x, y) \in [-1, 1]^2$ on a 128×128 raster grid. Each grid cell is indexed by row $r \in \{0, \dots, 127\}$ and column $c \in \{0, \dots, 127\}$. The pixel-to-world map is

$$\text{pix}^{-1}(r, c) = \left(\frac{2c}{127} - 1, 1 - \frac{2r}{127} \right) \in [-1, 1]^2,$$

so column c is the x -axis (left $\rightarrow$ right) and row r is the y -axis (top $\rightarrow$ bottom, sign-flipped to match image convention). Total pixels $|\Omega| = 128^2 = 16\,384$.

The full discrete action space has size $|\Theta| \cdot |\Omega| = 36 \cdot 16\,384 = 589\,824$.

1.1.2 Validity and reward

Define the **Inner Fit Polygon** at rotation θ as the locus of valid centroid positions:

$$\text{IFP}(F, P, \theta) = \{(x, y) \in \mathbb{R}^2 : P_{\theta,x,y} \subseteq F\}.$$

Equivalently, $\text{IFP}(F, P, \theta) = F \ominus R_\theta(P)$ (Minkowski erosion). It is well-defined and computable for any F (convex or concave) and convex P .

Contact and reward functions For a free-space polygon F and a placed-part polygon $Q \subseteq F$, define the **contact score**

$$\text{contact}(F, Q) = \alpha \text{adherence}(F, Q) + \beta \text{proximity}(F, Q) + \gamma,$$

where each term is normalized to lie in $[0, 1]$:

- **Adherence.** Fraction of Q 's perimeter that lies within ε of F 's boundary:

$$\text{adherence}(F, Q) = \frac{\mathcal{H}^1(\{x \in \partial Q : \text{dist}(x, \partial F) \leq \varepsilon\})}{\mathcal{H}^1(\partial Q)},$$

where $\mathcal{H}^1$ is 1-dimensional Hausdorff measure (arc-length) and ∂ denotes polygon boundary. We use $\varepsilon = 5 \cdot 10^{-3}$ in normalized $[-1, 1]$ coordinates.

- **Proximity.** Mean closeness of Q 's perimeter to ∂F , with an exponential decay:

$$\text{proximity}(F, Q) = \frac{1}{\mathcal{H}^1(\partial Q)} \int_{\partial Q} \exp\left(-\frac{\text{dist}(x, \partial F)}{\tau}\right) d\mathcal{H}^1(x),$$

with characteristic length $\tau = 0.05$.

Weights $(\alpha, \beta, \gamma) = (0.5, 0.2, 0.3)$ are fixed; defined in `src/geometry/rewards.py::compute_reward`. By construction $\text{contact}(F, Q) \in [\gamma, \gamma + \alpha + \beta] = [0.3, 1.0]$ for any $Q \subseteq F$.

The **scalar reward** at a placement is

$$r(F, P, \theta, x, y) = \begin{cases} \exp(k \cdot \text{contact}(F, P_{\theta, x, y})) & \text{if } (x, y) \in \text{IFP}(F, P, \theta) \\ 0 & \text{otherwise} \end{cases}$$

where $k = 10$ is a sharpening constant. The exponentiation turns additive contact differences into multiplicative reward differences, so the softmax-normalized heatmap (Section 1.3.1) concentrates mass near the best-contact placements.

Reward heatmap H_θ For fixed (F, P, θ) , define the discrete reward heatmap as the function

$$H_\theta : \{0, \dots, 127\}^2 \rightarrow \mathbb{R}_{\geq 0}, \quad H_\theta[r, c] = r(F, P, \theta, \text{pix}^{-1}(r, c)).$$

Concretely, $H_\theta[r, c]$ is the reward of placing $R_\theta(P)$ with its centroid at world coordinates

$$\text{pix}^{-1}(r, c) = \left(\frac{2c}{127} - 1, 1 - \frac{2r}{127} \right) \in [-1, 1]^2.$$

Properties:

property	value
Tensor shape	128×128
Support	$\text{rasterized IFP}(F, P, \theta) \cap \text{pix}^{-1}(\{0, \dots, 127\}^2)$
Range on support	$[\exp(0.3k), \exp(k)] = [\exp(3), \exp(10)] \approx [20, 22\,026]$
Range outside support	$\{0\}$
Stacked over 36 rotations	$\mathbf{H} \in \mathbb{R}_{\geq 0}^{36 \times 128 \times 128}$

The full per-pair reward tensor is $\mathbf{H} = (H_{\theta_0}, \dots, H_{\theta_{35}})$, computed once per pair in the precompute stage and stored as float16 in the training pkl. The brute-force optimum is $r^* = \max_{i, r, c} \mathbf{H}[i, r, c]$.

1.1.3 Optimization objective

The brute-force optimum for a pair is

$$(\theta^*, x^*, y^*) = \arg \max_{(\theta, x, y) \in \Theta \times \Omega} r(F, P, \theta, x, y), \quad r^* = r(F, P, \theta^*, x^*, y^*).$$

Computing it requires evaluating Shapely on every valid pixel — too expensive for inference. The placement network π_ϕ instead predicts a near-optimal placement in a single forward pass per rotation.

1.1.4 Quality metric

For a predicted placement $(\hat{\theta}, \hat{x}, \hat{y})$:

$$\text{recovery} = \frac{r(F, P, \hat{\theta}, \hat{x}, \hat{y})}{r^*} \in [0, 1].$$

We report mean recovery on a 2200-pair validation set.

1.2 Model

1.2.1 Inputs

For a given query rotation θ_i :

- $\mathbf{m}_F \in \{0, 1\}^{128 \times 128}$: rasterized free-space mask.
- $\mathbf{m}_{P, \theta_i} \in \{0, 1\}^{128 \times 128}$: rasterized mask of $R_{\theta_i}(P)$, with its centroid placed at the image center.

Stacked: $\mathbf{x}_i = \text{stack}(\mathbf{m}_F, \mathbf{m}_{P, \theta_i}) \in \mathbb{R}^{2 \times 128 \times 128}$.

1.2.2 Architecture: π_ϕ

A 4-level U-Net (`_SmallUNet` in `src/models/neural_bo_policy.py`):

$$\pi_\phi : \mathbb{R}^{2 \times 128 \times 128} \rightarrow \mathbb{R}^{128 \times 128}.$$

Building blocks Let $C \in \mathbb{R}^{c_{\text{in}} \times H \times W}$ denote a tensor with c_{in} channels at spatial resolution $H \times W$.

(a) **ConvBlock** $\mathcal{C}_{c_{\text{out}}}$. Two stacked stages of (3×3 conv with same-padding, BatchNorm2d, ReLU):

$$\mathcal{C}_{c_{\text{out}}}(C) = \text{ReLU}(\text{BN}(W_2 \star \text{ReLU}(\text{BN}(W_1 \star C)))) ,$$

where $W_1 \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}} \times 3 \times 3}$, $W_2 \in \mathbb{R}^{c_{\text{out}} \times c_{\text{out}} \times 3 \times 3}$, and $\star$ denotes 2D convolution with padding 1 (output preserves H, W).

(b) **Down**: max-pool with stride 2:

$$\text{Down}(C) \in \mathbb{R}^{c_{\text{in}} \times H/2 \times W/2}, \quad \text{Down}(C)[c, i, j] = \max_{a, b \in \{0, 1\}} C[c, 2i + a, 2j + b].$$

(c) **Up**: bilinear upsample by factor 2:

$$\text{Up}(C) \in \mathbb{R}^{c_{\text{in}} \times 2H \times 2W}.$$

(d) **SkipCat**: channel-wise concatenation with the corresponding encoder feature at matched resolution:

$$\text{SkipCat}(U, E) = [U; E] \in \mathbb{R}^{(c_U + c_E) \times H \times W}.$$

Forward pass With base width $b = 32$:

Encoder.

$$\begin{aligned} e_1 &= \mathcal{C}_b(\mathbf{x}_i) && \in \mathbb{R}^{32 \times 128 \times 128} \\ e_2 &= \mathcal{C}_{2b}(\text{Down}(e_1)) && \in \mathbb{R}^{64 \times 64 \times 64} \\ e_3 &= \mathcal{C}_{4b}(\text{Down}(e_2)) && \in \mathbb{R}^{128 \times 32 \times 32} \\ e_4 &= \mathcal{C}_{8b}(\text{Down}(e_3)) && \in \mathbb{R}^{256 \times 16 \times 16} \end{aligned}$$

Bottleneck.

$$g = \mathcal{C}_{8b}(e_4) \in \mathbb{R}^{256 \times 16 \times 16}.$$

Decoder. (mirror of encoder, with skip concatenation)

$$\begin{aligned} d_3 &= \mathcal{C}_{4b}(\text{SkipCat}(\text{Up}(g), e_3)) && \in \mathbb{R}^{128 \times 32 \times 32} \\ d_2 &= \mathcal{C}_{2b}(\text{SkipCat}(\text{Up}(d_3), e_2)) && \in \mathbb{R}^{64 \times 64 \times 64} \\ d_1 &= \mathcal{C}_b(\text{SkipCat}(\text{Up}(d_2), e_1)) && \in \mathbb{R}^{32 \times 128 \times 128} \end{aligned}$$

Output head. 1×1 convolution to a single channel:

$$\mathbf{z}_i = W_{\text{out}} \star d_1 \in \mathbb{R}^{1 \times 128 \times 128},$$

with $W_{\text{out}} \in \mathbb{R}^{1 \times 32 \times 1 \times 1}$. Squeezing the singleton channel gives $\mathbf{z}_i \in \mathbb{R}^{128 \times 128}$, the per-pixel logit map for rotation θ_i .

Receptive field and parameter count Receptive field at the bottleneck spans the full input (e_4 has spatial 16×16 covering 128×128 – each bottleneck cell “sees” all of the input through the 4-level downsampling). This is necessary because the optimum placement depends on global free-space geometry, not just local pixel neighborhoods.

Parameter table (with $b = 32$, $K = 9$ being the kernel-element count for a 3×3 conv):

stage	params (approx)
$\mathcal{C}_b$ on $c_{\text{in}} = 2$	$2 \cdot b \cdot K + b \cdot b \cdot K = 9\,792$
$\mathcal{C}_{2b}$ on b	$b \cdot 2b \cdot K + 2b \cdot 2b \cdot K = 55\,296$
$\mathcal{C}_{4b}$ on $2b$	$2b \cdot 4b \cdot K + 4b \cdot 4b \cdot K = 221\,184$
$\mathcal{C}_{8b}$ on $4b$	$4b \cdot 8b \cdot K + 8b \cdot 8b \cdot K = 884\,736$
Bottleneck $\mathcal{C}_{8b}$ on $8b$	$8b \cdot 8b \cdot K + 8b \cdot 8b \cdot K = 1\,179\,648$
Decoder $\mathcal{C}_{4b}$ on $8b + 4b$	$12b \cdot 4b \cdot K + 4b \cdot 4b \cdot K = 663\,552$
Decoder $\mathcal{C}_{2b}$ on $4b + 2b$	$6b \cdot 2b \cdot K + 2b \cdot 2b \cdot K = 165\,888$
Decoder $\mathcal{C}_b$ on $2b + b$	$3b \cdot b \cdot K + b \cdot b \cdot K = 36\,864$
Output 1×1 conv: $b \rightarrow 1$	$32 + 1 = 33$
BatchNorm affines + biases (small)	$\sim 6\,000$
Total	$\approx 3.13 \times 10^6$

1.2.3 Why this conditioning works

The model never has to *infer* rotation from the un-rotated part. Conditioning the input on θ_i via $\mathbf{m}_{P, \theta_i}$ collapses the prediction task into a translation-equivariant question: “given this oriented part and this free space, where does it fit best?” This is the standard inductive bias of a U-Net.

1.3 Training objective

1.3.1 Target distribution

For each (F, P, θ_i) training example, define the *normalized soft target*

$$\tilde{H}_{\theta_i}[r, c] = \frac{H_{\theta_i}[r, c]}{\sum_{r', c'} H_{\theta_i}[r', c']}.$$

Since the precomputed H_{θ_i} uses $r = \exp(k \cdot \text{contact})$, $\tilde{H}_{\theta_i}$ is a temperature- $1/k$ softmax of the underlying contact field over the IFP.

1.3.2 Loss

Let $\mathbf{p}_i = \text{softmax}(\mathbf{z}_i) \in \Delta^{16383}$ (softmax over the 16 384 flattened pixels). The per-example soft cross-entropy loss is

$$\mathcal{L}_i(\phi) = - \sum_{r,c} \tilde{H}_{\theta_i}[r, c] \log \mathbf{p}_i[r, c].$$

Equivalently, $\mathcal{L}_i = D_{\text{KL}}(\tilde{H}_{\theta_i} \parallel \mathbf{p}_i) + H(\tilde{H}_{\theta_i})$, where the entropy term is an example-dependent constant.

Compared to hard-label CE (one-hot at $\arg \max$), the soft loss provides gradient on every cell weighted by reward, which empirically lifts val recovery from $0.43 \rightarrow 0.72$ in our experiments.

1.3.3 Training-step distribution

Let $\mathcal{D} = \{(F_j, P_j, H^{(j)})\}_{j=1}^N$ be the corpus of $N = 22\,000$ training pairs (each with its 36-rotation heatmap tensor). Sampling for each SGD step:

$$j \sim \text{Uniform}\{1, \dots, N\}, \quad i \sim \text{Uniform}\{0, \dots, 35\},$$

and the per-step empirical objective is $\mathbb{E}_{j,i}[\mathcal{L}_i(\phi; F_j, P_j)]$. Roughly $N \cdot 36 = 7.92 \times 10^5$ distinct (j, i) examples are addressable.

1.3.4 Augmentation: dihedral group D_2

Applied at the dataloader level. Let σ_h (horizontal flip) and σ_v (vertical flip) act on the rasterized image plane. Each is applied with probability $\frac{1}{2}$, independently:

- σ_h : spatial axis-1 reversal; rotation reindex $i \mapsto (-i) \bmod 36$.
- σ_v : spatial axis-0 reversal; rotation reindex $i \mapsto (18 - i) \bmod 36$.

The transform is applied jointly to $(\mathbf{m}_F, \mathbf{m}_{P, \theta_i}, \tilde{H}_{\theta_i})$, preserving the input–target consistency. Under reflection of the entire scene, the placement problem maps to itself, so the loss is unchanged in distribution. Effective dataset multiplier: $|D_2| = 4$.

1.3.5 Optimization

- Optimizer: AdamW, weight decay 10^{-4} .
- Learning rate: $\eta_0 = 3 \times 10^{-4}$, cosine-annealed to 0 over $T = 8000$ steps.
- Batch size: 256.
- Mixed precision (FP16 amp).
- Hardware: Modal A100-80GB, ~ 4.2 steps/sec, ~ 30 min wall.

1.4 Inference

For a deployment query on (F, P) :

1. Rasterize $\mathbf{m}_F$.
2. For each $i \in \{0, \dots, 35\}$: rasterize $\mathbf{m}_{P, \theta_i}$, run $\mathbf{z}_i = \pi_\phi(\mathbf{x}_i)$.
3. Stack: $\mathbf{Z} = [\mathbf{z}_0, \dots, \mathbf{z}_{35}] \in \mathbb{R}^{36 \times 128 \times 128}$.
4. Predict $(\hat{i}, \hat{r}, \hat{c}) = \arg \max_{i,r,c} \mathbf{Z}[i, r, c]$, giving $\hat{\theta} = \theta_i$, $(\hat{x}, \hat{y}) = \text{pix}^{-1}(\hat{r}, \hat{c})$.

5. **Optional refinement.** Evaluate Shapely reward on a local window

$$\mathcal{W} = \{(\theta_{i+\delta_\theta}, \hat{r} + \delta_r, \hat{c} + \delta_c) : |\delta_\theta| \leq K, |\delta_r|, |\delta_c| \leq N\}$$

and return $\arg \max_{\mathcal{W}} r(\cdot)$. With $N = 10, K = 2, |\mathcal{W}| = 5 \cdot 21^2 = 2205$ Shapely calls at $\sim 75 \mu s$ each ≈ 165 ms. Closes the model-vs-brute-force gap.

Latency without refinement: 36 forward passes on a 3M-param U-Net, dominated by per-rotation Shapely-free rasterization ~ 30 ms total on a CPU; the pure model forward passes are < 50 ms on GPU.

1.5 Data pipeline

1.5.1 Pair generation

- **Convex-convex.** Random convex hulls from n uniform points; 12 000 pairs sampled by `generate_bc_dataset.py`.
- **Concave-fs / convex-part.** From `bo_train_pool_10k.pkl`: 10 000 pairs with non-convex F (random star polygons) and convex P .

1.5.2 IFP precompute

For each pair and each θ_i : compute $\text{IFP}(F, P, \theta_i)$ via Minkowski erosion using `pyclipper` (`src/geometry/ifp.py`). Rasterize to $\{0, 1\}^{128 \times 128}$.

1.5.3 Reward heatmap precompute

For each pixel inside the rasterized IFP, evaluate $r(F, P, \theta_i, x, y)$ via Shapely. Pixels outside IFP are 0. Stored as (36, 128, 128) float32 per pair.

Total Shapely queries: $\sim 7.1 \times 10^8$ for convex + $\sim 3.1 \times 10^8$ for concave $\approx 10^9$ queries. Executed in parallel across 100 Modal containers $\times$ 8 cores. Total wall ≈ 25 minutes; cost $\approx \$40$.

1.5.4 Rotated-part-mask precompute

For each pair: rasterize $R_{\theta_i}(P)$ centered at origin, for $i = 0, \dots, 35$. Stored as (36, 128, 128) uint8 per pair. Used as the second input channel at training and inference.

1.5.5 Combined corpus

$N_{\text{train}} = 19\,800$ pairs (10 800 convex + 9 000 concave), $N_{\text{val}} = 2\,200$ (1200 convex + 1000 concave). Combined pkl: 26.68 GB (fp16 heatmaps + uint8 masks).

1.6 Results

Model selection: best mean val recovery on a fixed 2200-pair validation set, evaluated every 250 steps. Compare model-only recovery (no Shapely refine):

Configuration	Val recovery
Hard label, hierarchical (tile + cell)	0.04
Soft label, hierarchical	0.43
Soft label, hierarchical, tile-only	0.32
Per-(pair, θ) U-Net, convex-only ($N = 10\,800$)	0.72
Per-(pair, θ) U-Net, combined ($N = 19\,800$)	0.73

The convex-only and combined models are nearly tied on a mixed val set; combined generalizes to concave free spaces from a single checkpoint. With Shapely refinement, both push to near-1.0 recovery (analytically: refinement window contains the true argmax with high probability if the model lands within ± 10 pixels and ± 2 rotation bins).

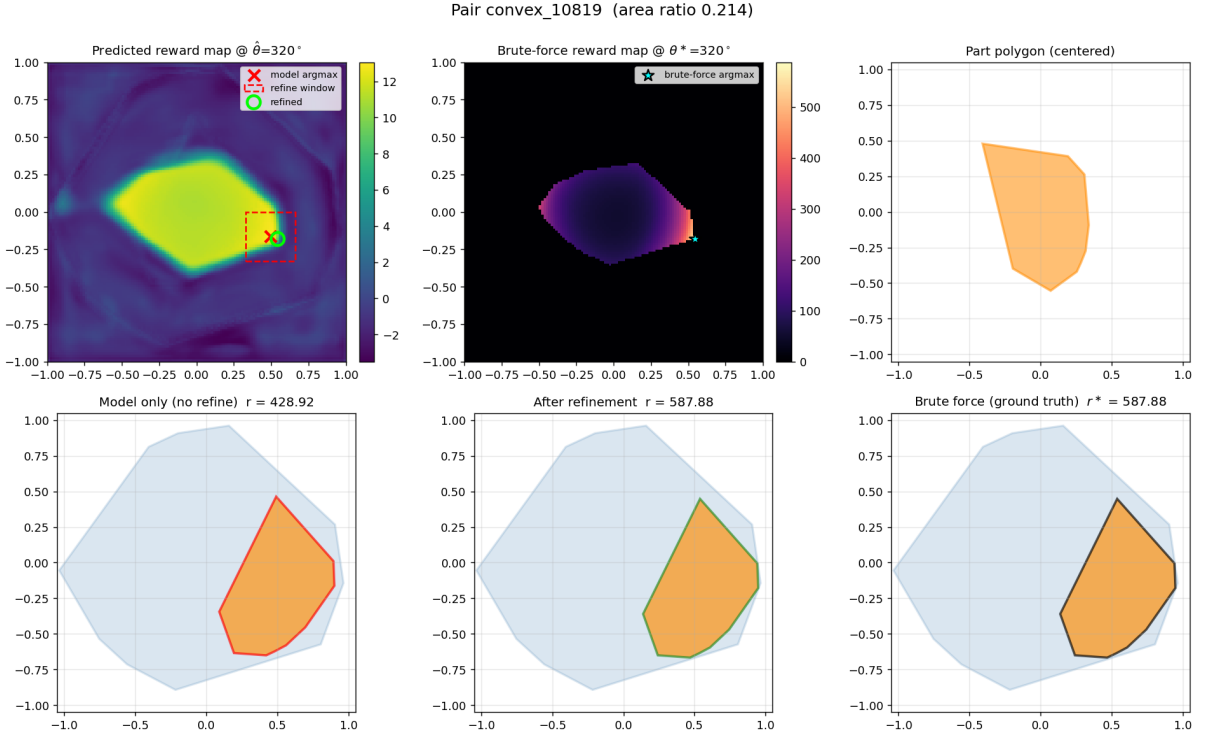
1.7 Implementation pointers

concern	file
Model	<code>src/models/neural_bo_policy.py::_SmallUNet</code>
Training script	<code>scripts/train_perthet.py</code>
Modal training entrypoint	<code>modal_train_perthet.py</code>
IFP exact (handles concave fs)	<code>src/geometry/ifp.py::compute_ifp_exact</code>
Reward function	<code>src/geometry/rewards.py::compute_reward_exp</code>
Concave precompute	<code>modal_concave_precompute.py</code>
Combined-pkl build	<code>modal_build_combined.py</code>
Best ckpt — convex	<code>checkpoints/perthet/final.pt</code>
Best ckpt — combined	<code>checkpoints/perthet_combined/final.pt</code>

2 Examples — Convex-convex pairs

The free-space polygon and the part polygon are both convex. Brute-force ground truth r^* comes from precomputed heatmap chunks (`data/reward_heatmaps_exp_k10_inside.npy_chunks/`). For each pair we show, top row left to right: the model’s predicted reward map at $\hat{\theta}$ (with the refinement window outlined and the refined target circled), the brute-force ground-truth reward map at θ^* , and the part polygon (centered). Bottom row: the model-only placement, the placement after Shapely refinement, and the brute-force placement.

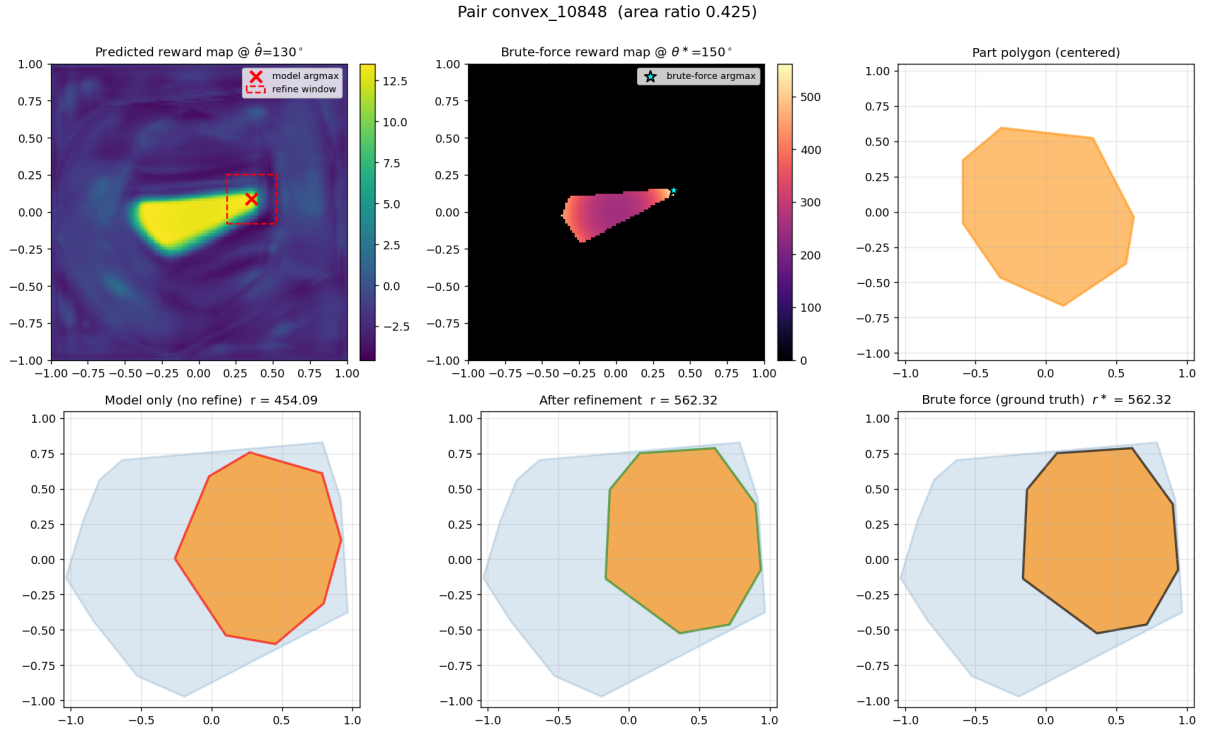
2.1 Pair convex_10819 (area ratio 0.214)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	320	(+0.543, -0.181)	587.882	1.000
Model only	320	(+0.496, -0.165)	428.924	0.730
Refined (± 10 px, $\pm 2\theta$ -bins)	320	(+0.543, -0.181)	587.882	1.000

Cost	Value
Rasterize (36 rotated part masks)	8.4 ms
Model forward (36 passes, batched)	861.0 ms
Shapely refinement (2205 calls, target 2205)	730.1 ms
Total	1599.4 ms

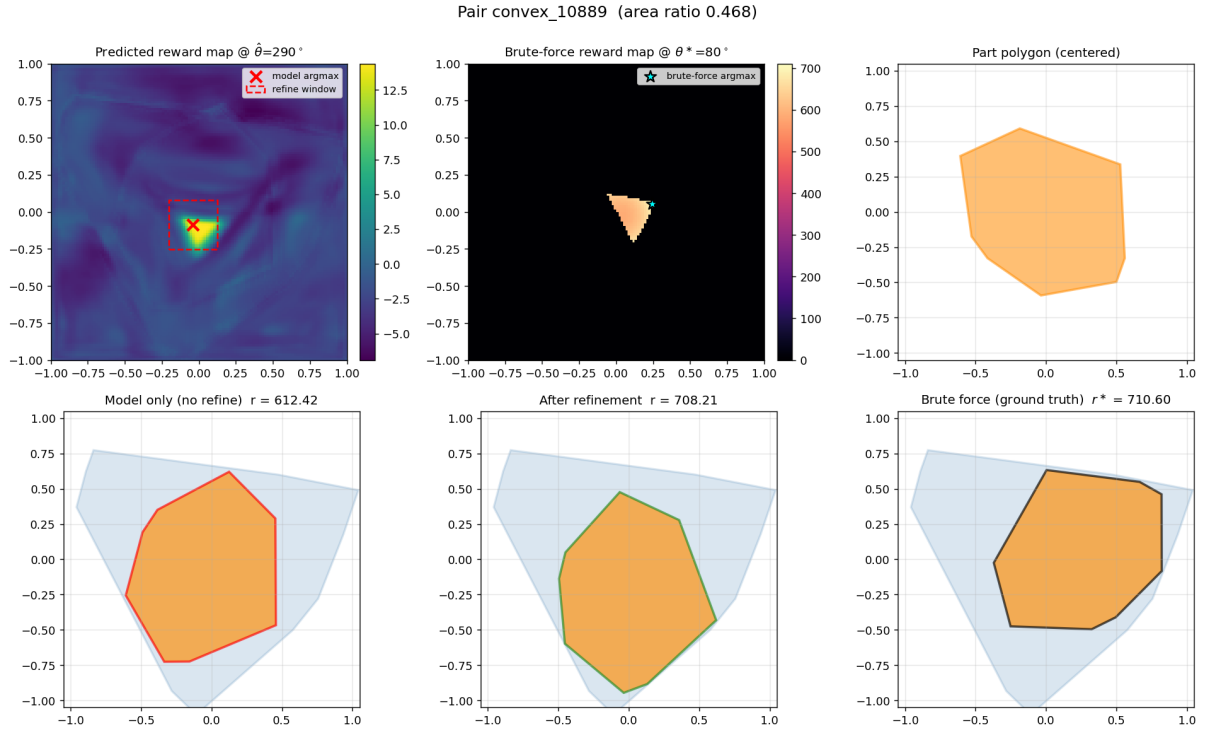
2.2 Pair convex_10848 (area ratio 0.425)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	150	(+0.386, +0.150)	562.316	1.000
Model only	130	(+0.354, +0.087)	454.095	0.808
Refined (± 10 px, ± 2 θ -bins)	150	(+0.386, +0.150)	562.316	1.000

Cost	Value
Rasterize (36 rotated part masks)	7.4 ms
Model forward (36 passes, batched)	47.5 ms
Shapely refinement (2205 calls, target 2205)	481.0 ms
Total	535.9 ms

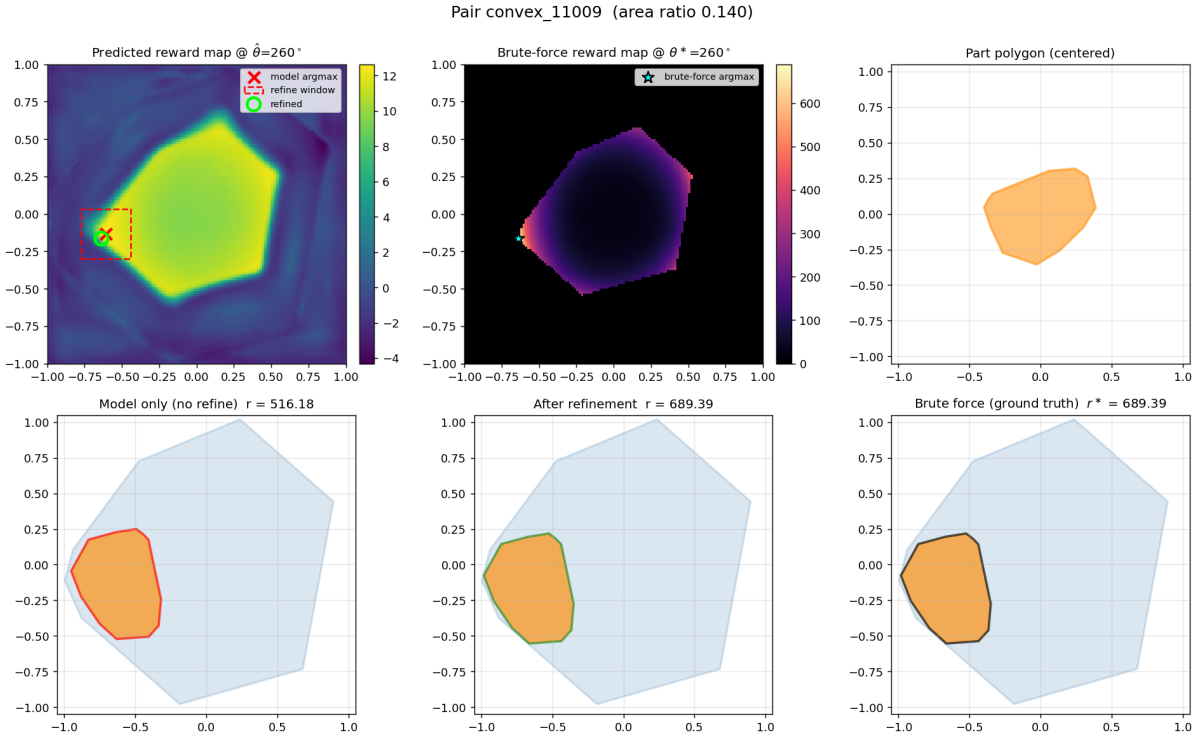
2.3 Pair convex_10889 (area ratio 0.468)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	80	(+0.244, +0.055)	710.603	1.000
Model only	290	(-0.039, -0.087)	612.424	0.862
Refined (± 10 px, ± 2 θ -bins)	310	(+0.024, -0.244)	708.210	0.997

Cost	Value
Rasterize (36 rotated part masks)	6.2 ms
Model forward (36 passes, batched)	45.2 ms
Shapely refinement (2205 calls, target 2205)	394.2 ms
Total	445.6 ms

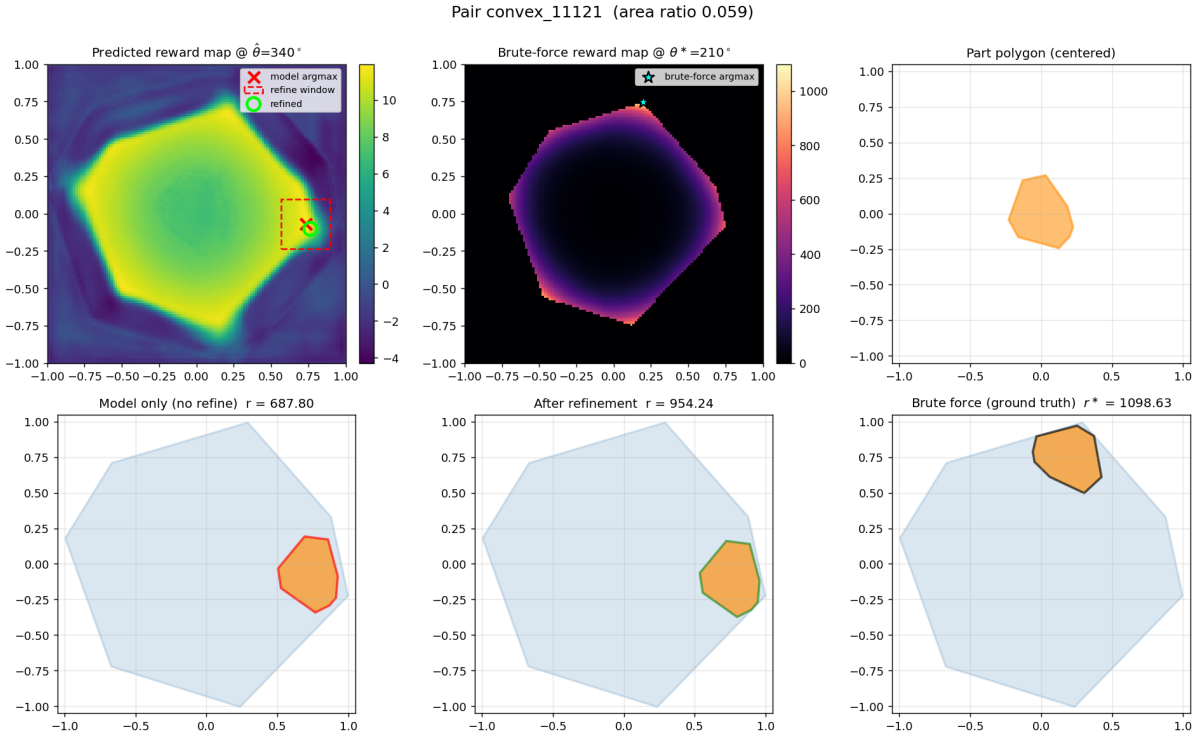
2.4 Pair convex_11009 (area ratio 0.140)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	260	$(-0.638, -0.165)$	689.395	1.000
Model only	260	$(-0.606, -0.134)$	516.184	0.749
Refined (± 10 px, $\pm 2 \theta$ -bins)	260	$(-0.638, -0.165)$	689.395	1.000

Cost	Value
Rasterize (36 rotated part masks)	6.7 ms
Model forward (36 passes, batched)	44.4 ms
Shapely refinement (2205 calls, target 2205)	700.8 ms
Total	752.0 ms

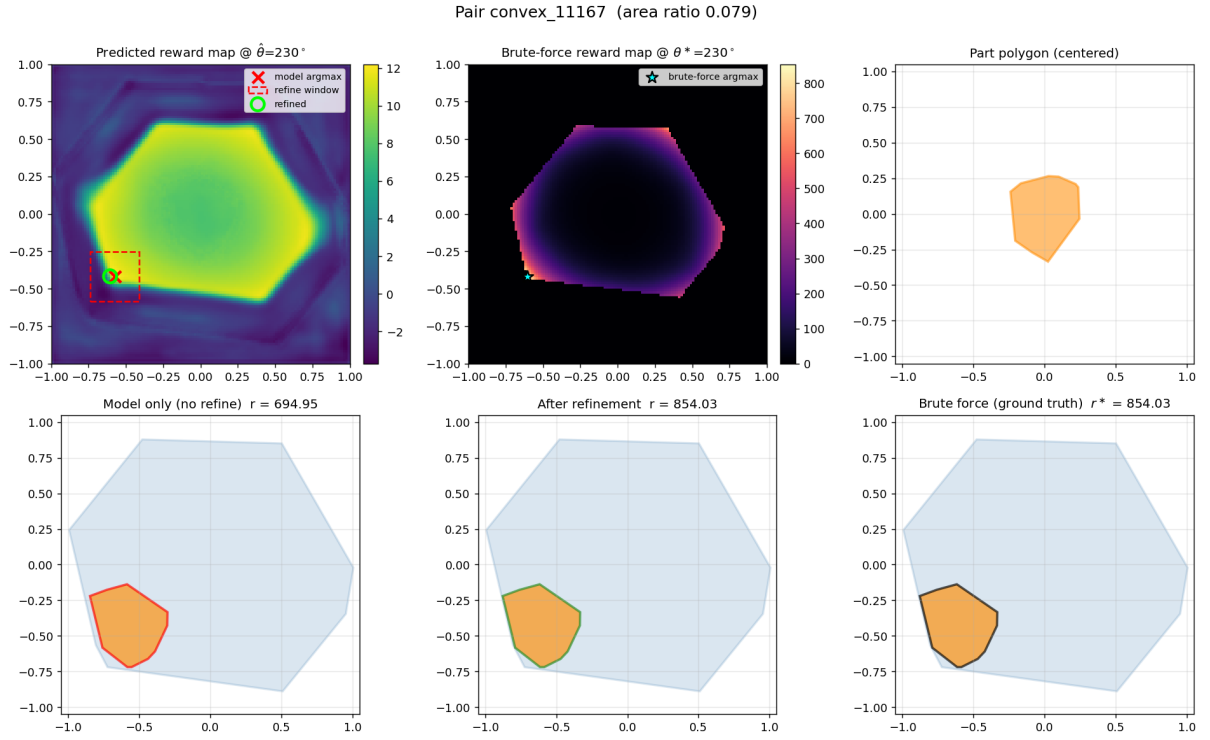
2.5 Pair convex_11121 (area ratio 0.059)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	210	(+0.197, +0.748)	1098.633	1.000
Model only	340	(+0.732, -0.071)	687.801	0.626
Refined (± 10 px, $\pm 2 \theta$ -bins)	340	(+0.764, -0.102)	954.235	0.869

Cost	Value
Rasterize (36 rotated part masks)	4.7 ms
Model forward (36 passes, batched)	45.6 ms
Shapely refinement (2205 calls, target 2205)	771.5 ms
Total	821.8 ms

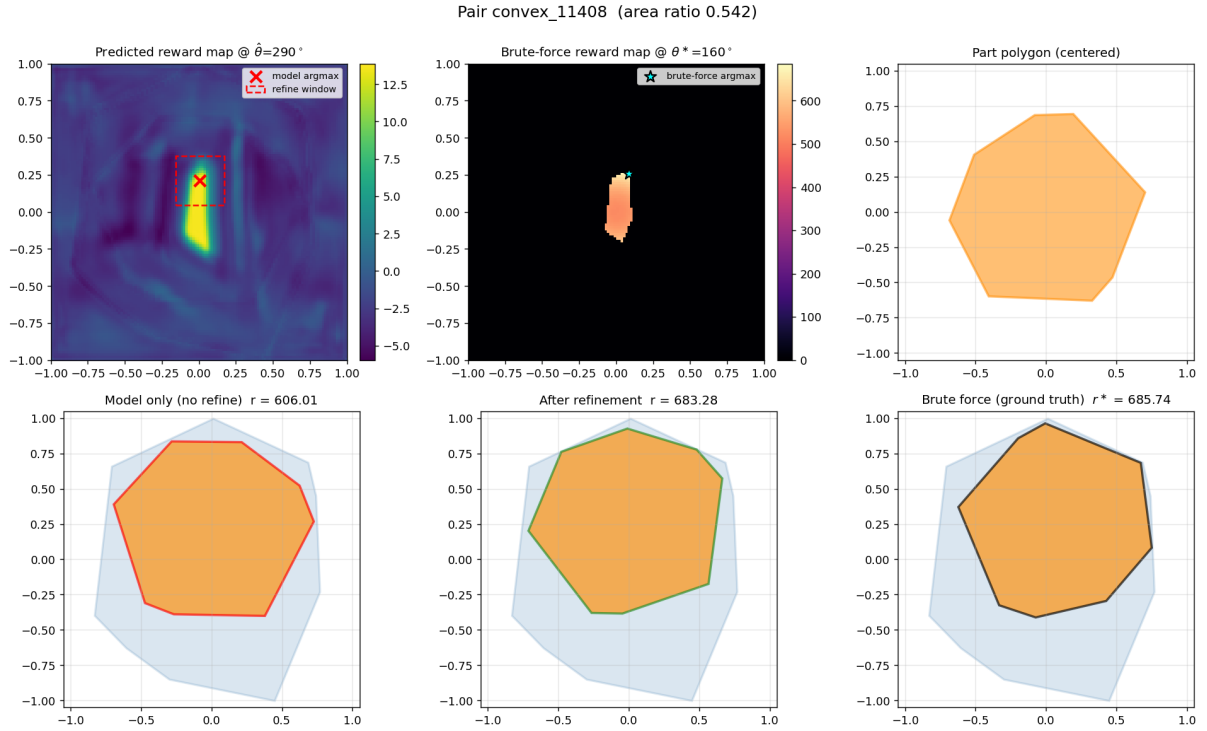
2.6 Pair convex_11167 (area ratio 0.079)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	230	$(-0.606, -0.417)$	854.025	1.000
Model only	230	$(-0.575, -0.417)$	694.950	0.814
Refined (± 10 px, ± 2 θ -bins)	230	$(-0.606, -0.417)$	854.025	1.000

Cost	Value
Rasterize (36 rotated part masks)	5.7 ms
Model forward (36 passes, batched)	44.8 ms
Shapely refinement (2205 calls, target 2205)	657.6 ms
Total	708.1 ms

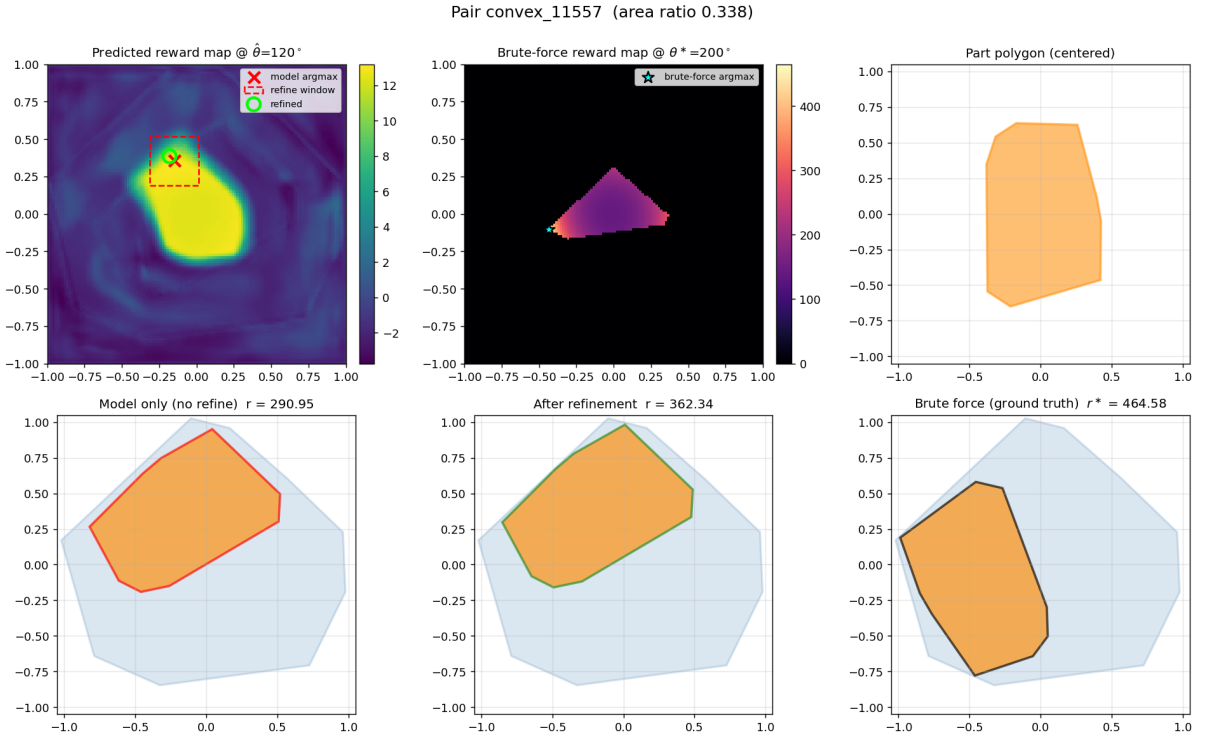
2.7 Pair convex_11408 (area ratio 0.542)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	160	(+0.087, +0.260)	685.735	1.000
Model only	290	(+0.008, +0.213)	606.011	0.884
Refined (± 10 px, ± 2 θ -bins)	310	(+0.008, +0.276)	683.282	0.996

Cost	Value
Rasterize (36 rotated part masks)	6.6 ms
Model forward (36 passes, batched)	44.7 ms
Shapely refinement (2205 calls, target 2205)	529.8 ms
Total	581.1 ms

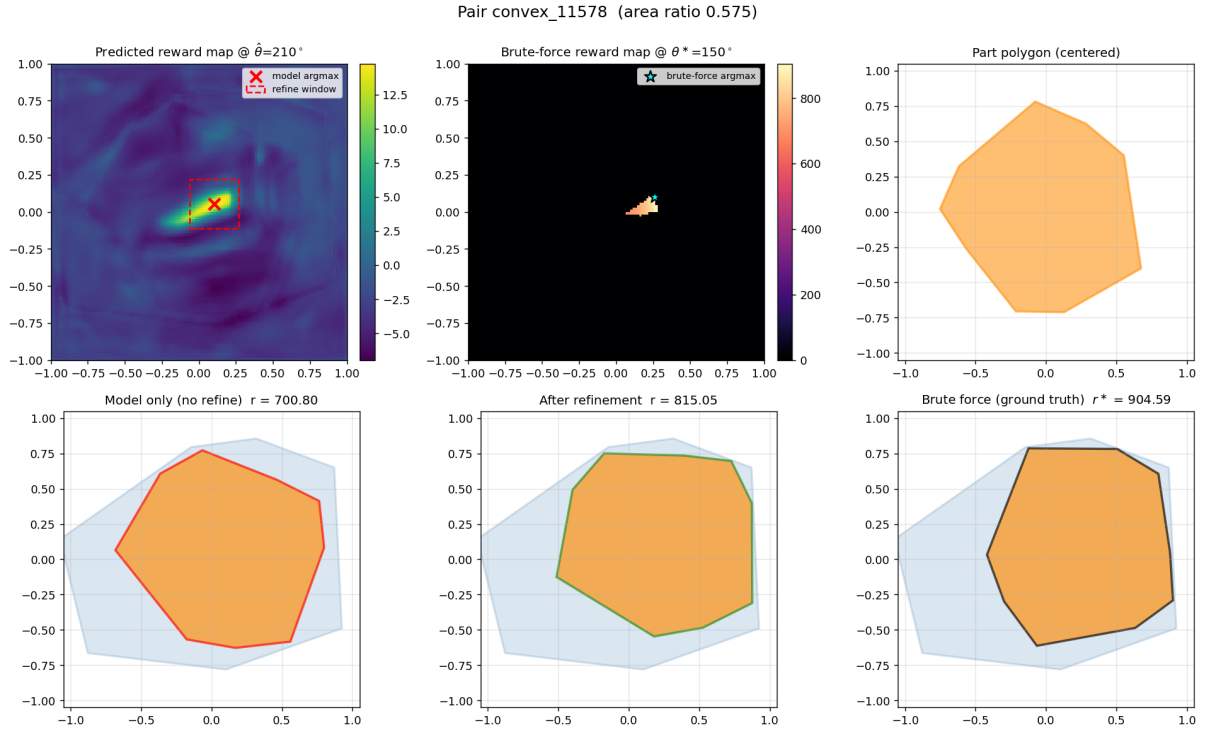
2.8 Pair convex_11557 (area ratio 0.338)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	200	$(-0.433, -0.102)$	464.579	1.000
Model only	120	$(-0.150, +0.354)$	290.952	0.626
Refined (± 10 px, $\pm 2 \theta$ -bins)	120	$(-0.181, +0.386)$	362.344	0.780

Cost	Value
Rasterize (36 rotated part masks)	6.3 ms
Model forward (36 passes, batched)	310.5 ms
Shapely refinement (2205 calls, target 2205)	615.3 ms
Total	932.1 ms

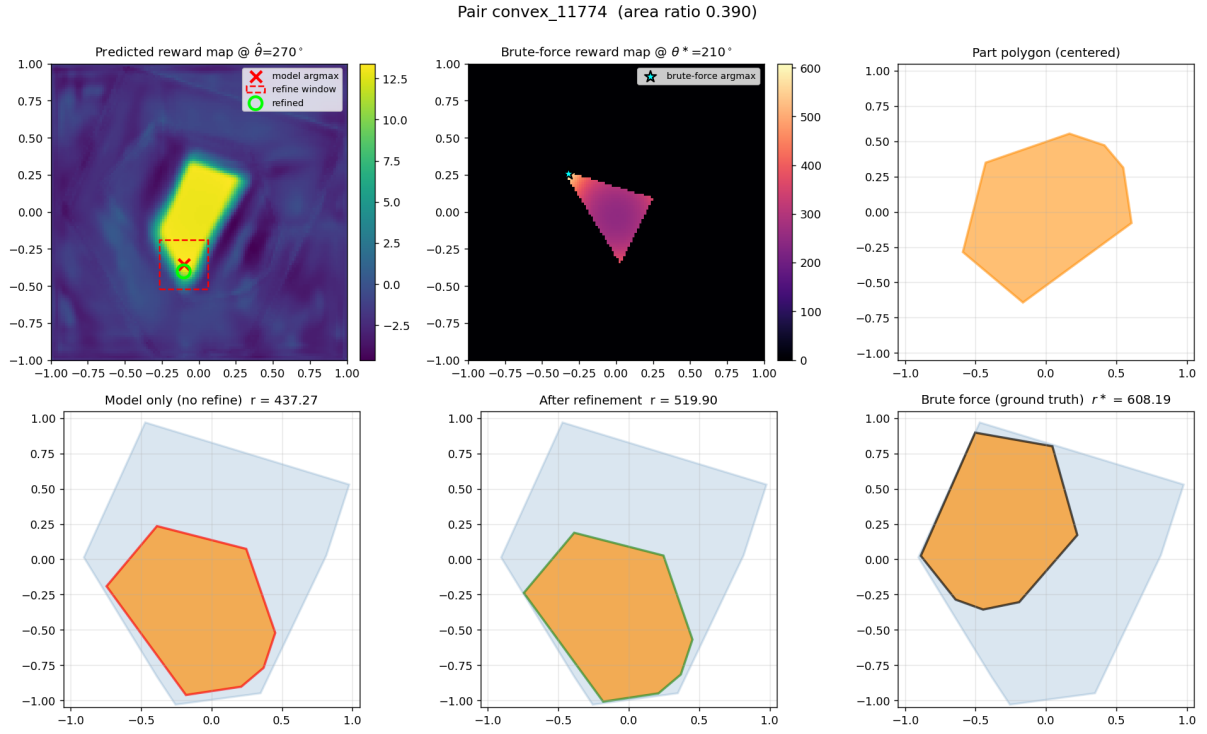
2.9 Pair convex_11578 (area ratio 0.575)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	150	(+0.260, +0.102)	904.592	1.000
Model only	210	(+0.102, +0.055)	700.800	0.775
Refined (± 10 px, ± 2 θ -bins)	230	(+0.228, +0.134)	815.048	0.901

Cost	Value
Rasterize (36 rotated part masks)	6.8 ms
Model forward (36 passes, batched)	44.0 ms
Shapely refinement (2205 calls, target 2205)	492.4 ms
Total	543.2 ms

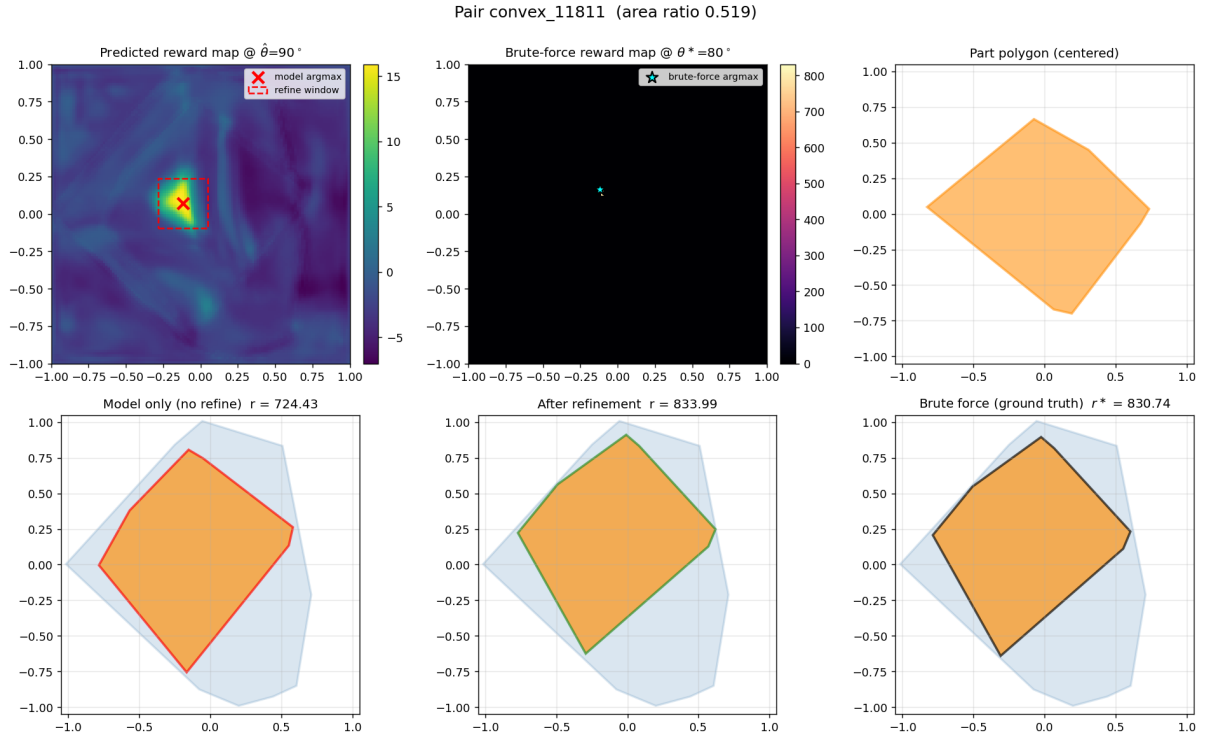
2.10 Pair convex_11774 (area ratio 0.390)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	210	$(-0.323, +0.260)$	608.192	1.000
Model only	270	$(-0.102, -0.354)$	437.271	0.719
Refined (± 10 px, ± 2 θ -bins)	270	$(-0.102, -0.402)$	519.902	0.855

Cost	Value
Rasterize (36 rotated part masks)	5.2 ms
Model forward (36 passes, batched)	87.1 ms
Shapely refinement (2205 calls, target 2205)	473.3 ms
Total	565.6 ms

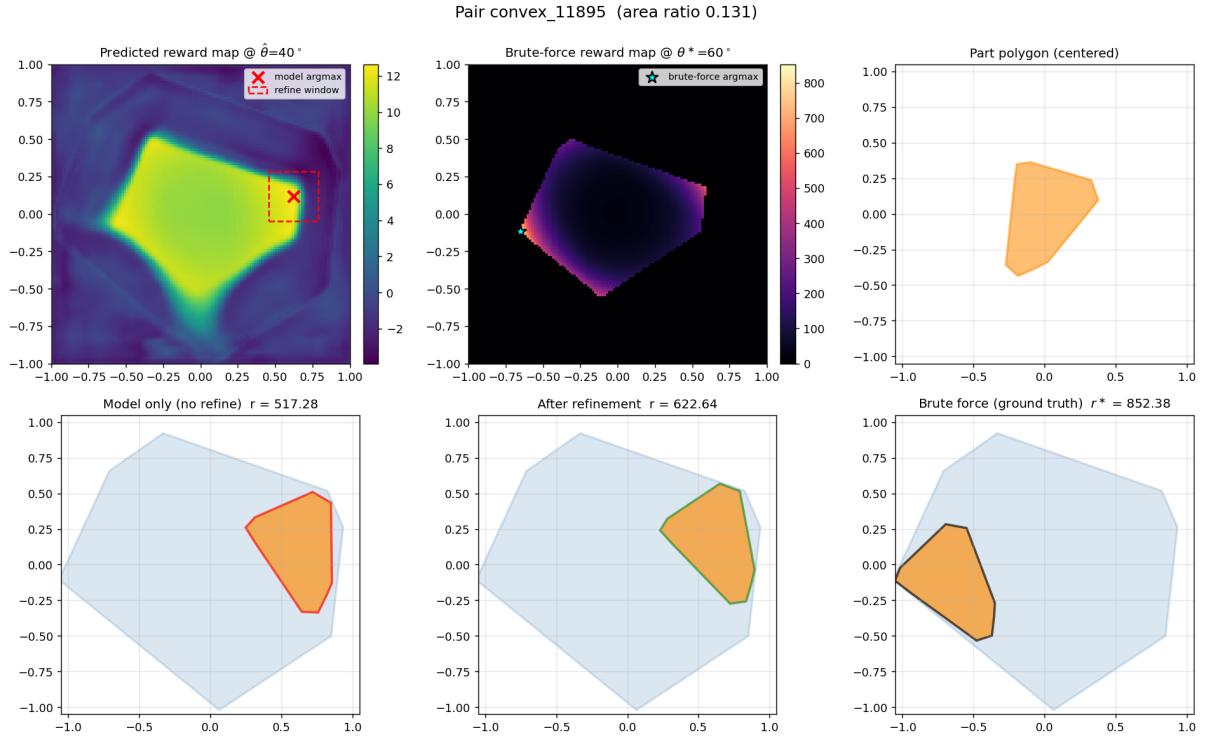
2.11 Pair convex_11811 (area ratio 0.519)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	80	$(-0.118, +0.165)$	830.735	1.000
Model only	90	$(-0.118, +0.071)$	724.427	0.872
Refined (± 10 px, $\pm 2 \theta$ -bins)	80	$(-0.102, +0.181)$	833.989	1.004

Cost	Value
Rasterize (36 rotated part masks)	7.6 ms
Model forward (36 passes, batched)	221.3 ms
Shapely refinement (2205 calls, target 2205)	345.8 ms
Total	574.7 ms

2.12 Pair convex_11895 (area ratio 0.131)



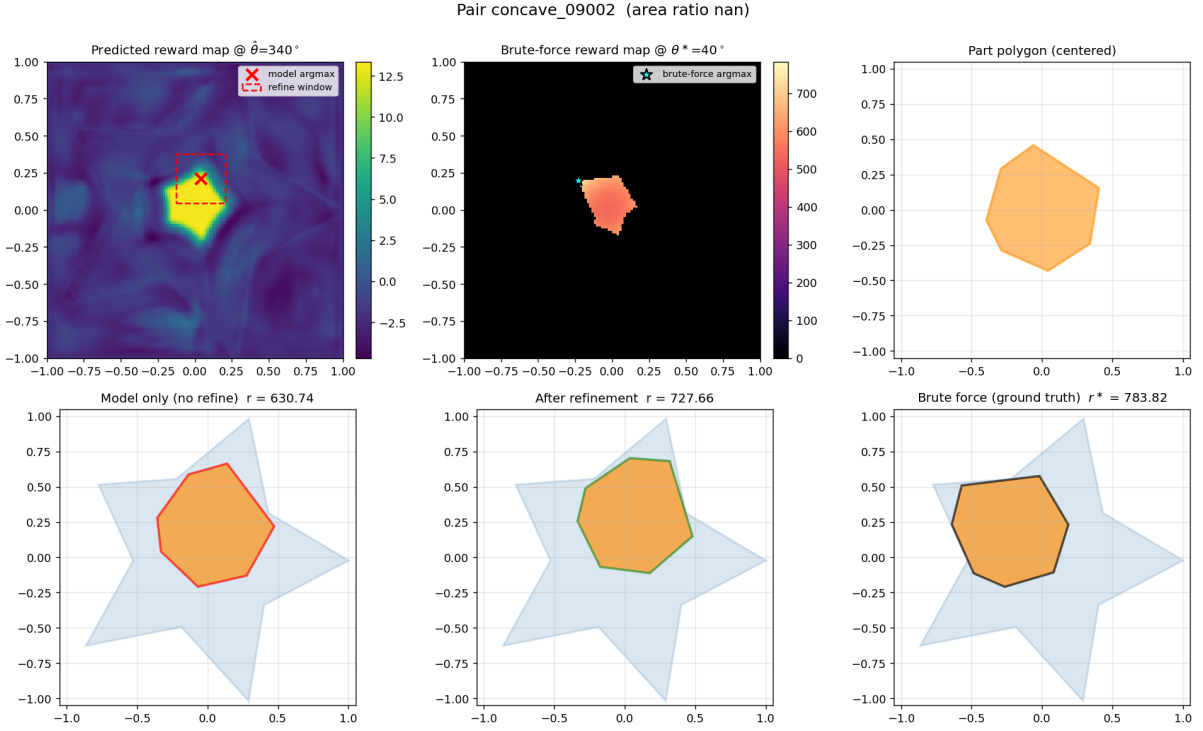
Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	60	$(-0.654, -0.118)$	852.383	1.000
Model only	40	$(+0.622, +0.118)$	517.275	0.607
Refined (± 10 px, $\pm 2 \theta$ -bins)	50	$(+0.622, +0.165)$	622.640	0.730

Cost	Value
Rasterize (36 rotated part masks)	5.0 ms
Model forward (36 passes, batched)	42.4 ms
Shapely refinement (2205 calls, target 2205)	626.1 ms
Total	673.6 ms

3 Examples — Concave free-space pairs

The free-space polygon is non-convex (random star polygon from `bo_train_pool_10k.pkl`); the part remains convex. The model and refinement use the same pipeline; brute-force r^* is computed on the fly via `compute_ifp_exact` + exhaustive Shapely scoring inside the IFP (Section 1.4). Same six-panel layout as before.

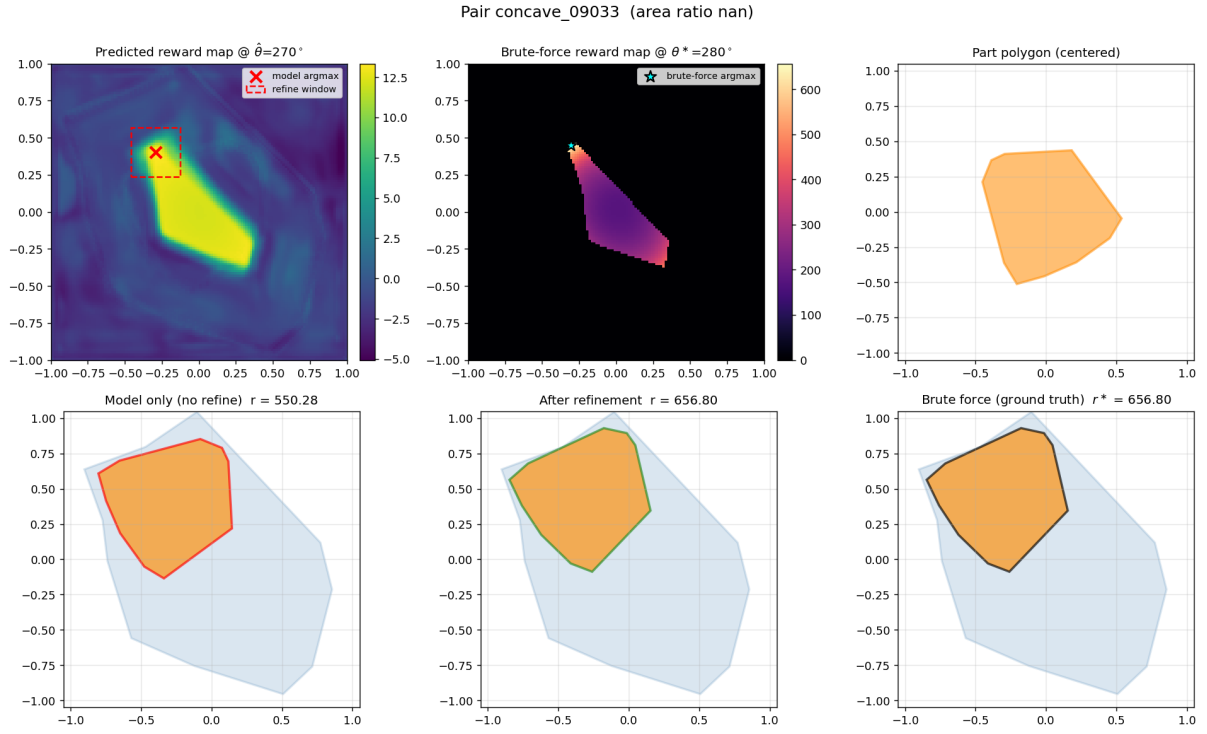
3.1 Pair concave_09002 (area ratio nan)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	40	$(-0.228, +0.197)$	783.819	1.000
Model only	340	$(+0.039, +0.213)$	630.739	0.805
Refined (± 10 px, $\pm 2 \theta$ -bins)	320	$(+0.071, +0.291)$	727.659	0.928

Cost	Value
Rasterize (36 rotated part masks)	6.1 ms
Model forward (36 passes, batched)	281.6 ms
Shapely refinement (2205 calls, target 2205)	631.7 ms
Total	919.4 ms

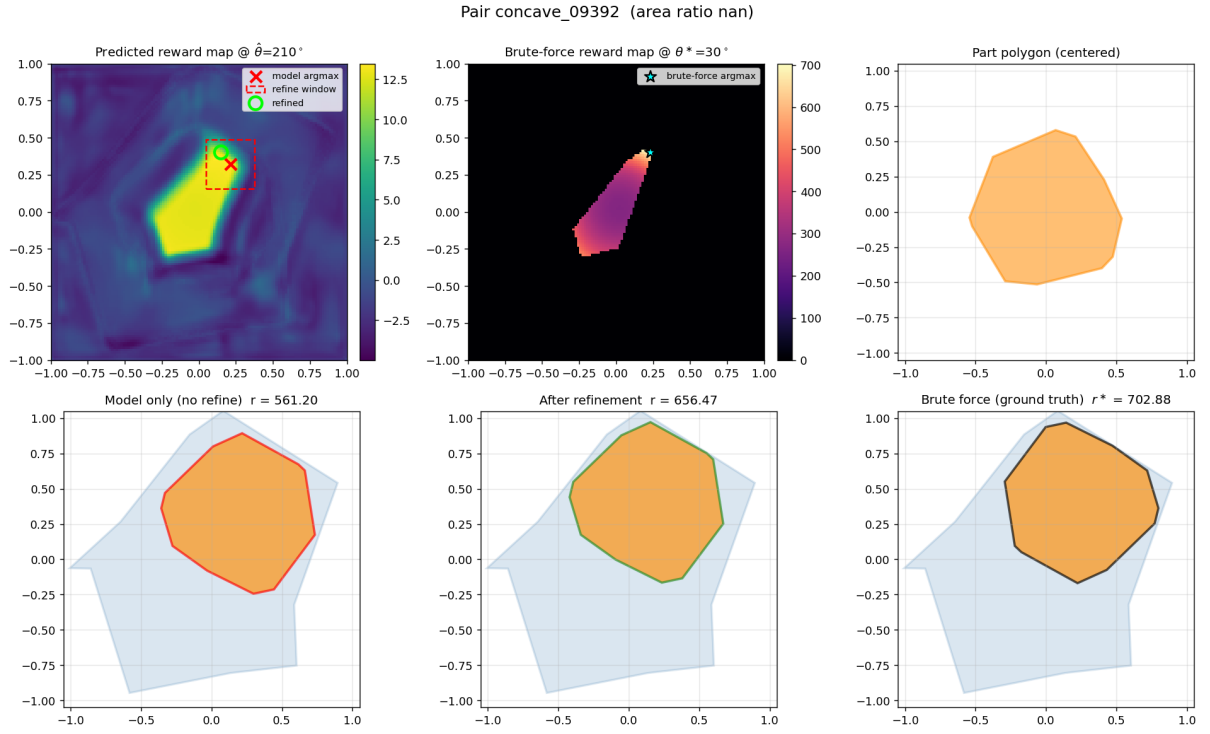
3.2 Pair concave_09033 (area ratio nan)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	280	$(-0.307, +0.449)$	656.799	1.000
Model only	270	$(-0.291, +0.402)$	550.279	0.838
Refined (± 10 px, ± 2 θ -bins)	280	$(-0.307, +0.449)$	656.798	1.000

Cost	Value
Rasterize (36 rotated part masks)	5.2 ms
Model forward (36 passes, batched)	404.5 ms
Shapely refinement (2205 calls, target 2205)	504.6 ms
Total	914.4 ms

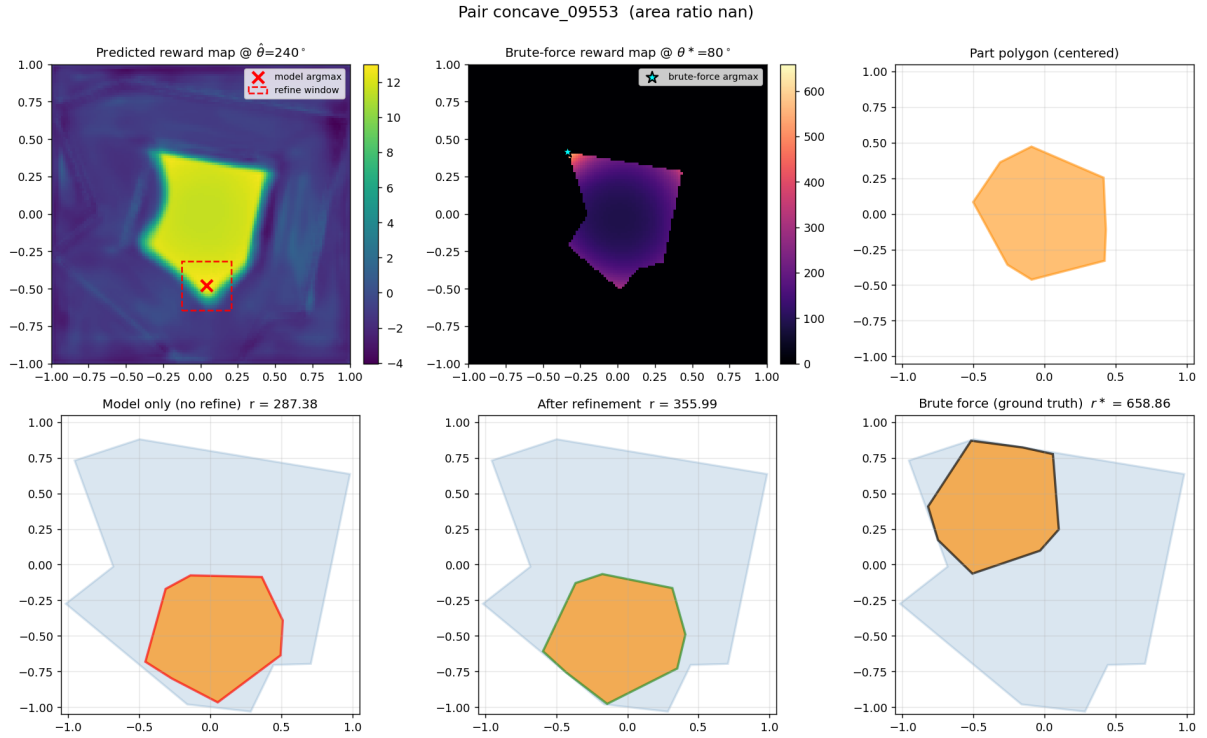
3.3 Pair concave_09392 (area ratio nan)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	30	(+0.228, +0.402)	702.877	1.000
Model only	210	(+0.213, +0.323)	561.201	0.798
Refined (± 10 px, $\pm 2 \theta$ -bins)	210	(+0.150, +0.402)	656.467	0.934

Cost	Value
Rasterize (36 rotated part masks)	6.7 ms
Model forward (36 passes, batched)	432.1 ms
Shapely refinement (2205 calls, target 2205)	626.4 ms
Total	1065.2 ms

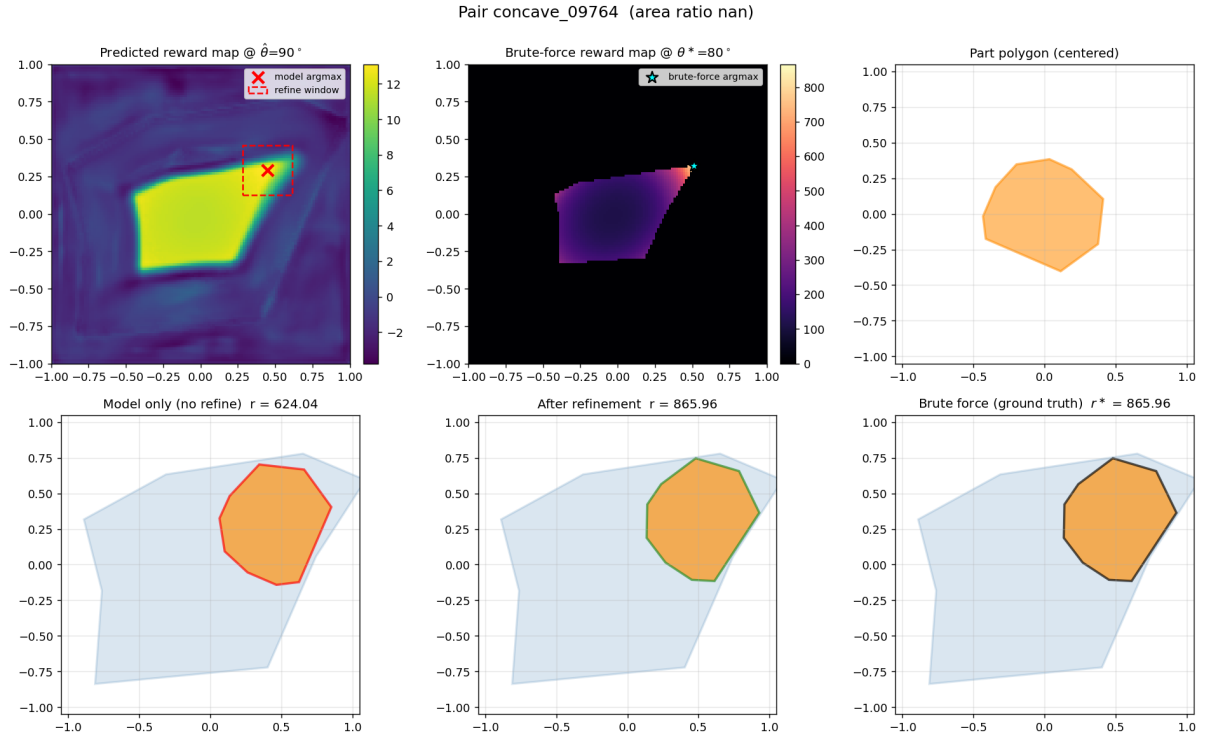
3.4 Pair concave_09553 (area ratio nan)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	80	$(-0.339, +0.417)$	658.856	1.000
Model only	240	$(+0.039, -0.480)$	287.381	0.436
Refined (± 10 px, ± 2 θ -bins)	230	$(-0.071, -0.496)$	355.985	0.540

Cost	Value
Rasterize (36 rotated part masks)	5.2 ms
Model forward (36 passes, batched)	265.8 ms
Shapely refinement (2205 calls, target 2205)	649.8 ms
Total	920.8 ms

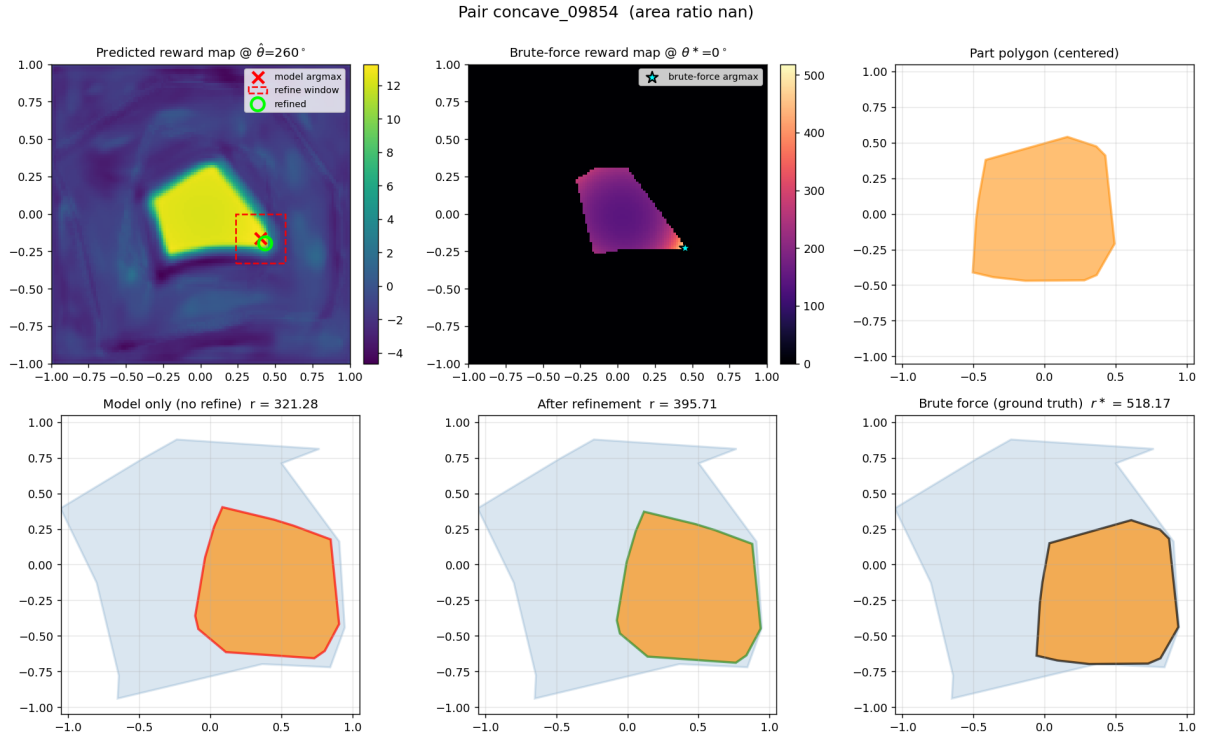
3.5 Pair concave_09764 (area ratio nan)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	80	(+0.512, +0.323)	865.964	1.000
Model only	90	(+0.449, +0.291)	624.041	0.721
Refined (± 10 px, ± 2 θ -bins)	80	(+0.512, +0.323)	865.964	1.000

Cost	Value
Rasterize (36 rotated part masks)	8.7 ms
Model forward (36 passes, batched)	348.8 ms
Shapely refinement (2205 calls, target 2205)	498.5 ms
Total	856.0 ms

3.6 Pair concave_09854 (area ratio nan)



Method	θ (deg)	(x, y)	Reward	Recovery
Brute force (r^*)	0	(+0.449, -0.228)	518.172	1.000
Model only	260	(+0.402, -0.165)	321.276	0.620
Refined (± 10 px, $\pm 2 \theta$ -bins)	260	(+0.433, -0.197)	395.707	0.764

Cost	Value
Rasterize (36 rotated part masks)	8.3 ms
Model forward (36 passes, batched)	250.2 ms
Shapely refinement (2205 calls, target 2205)	532.9 ms
Total	791.4 ms

4 Summary

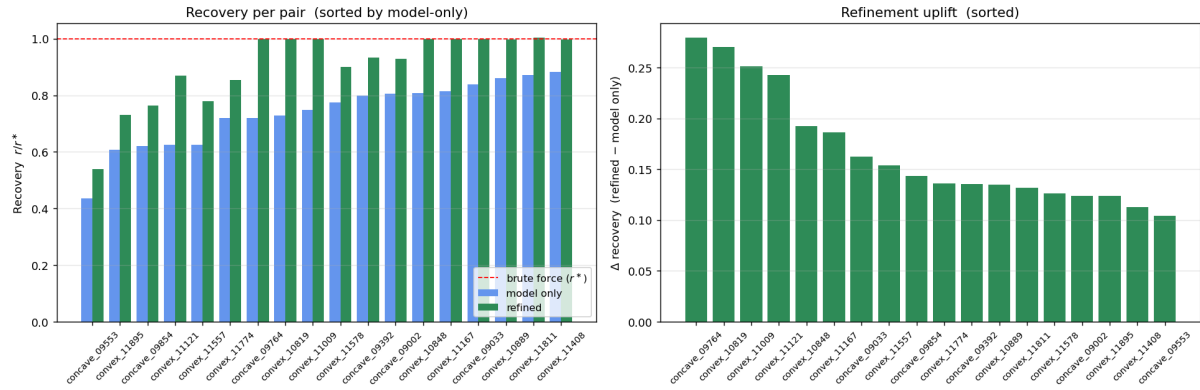


Figure 1: Left: recovery r/r^* per pair (model-only vs. refined). Right: refinement uplift sorted descending.

Statistic	Value
Pairs reported	18
Mean recovery (model only)	0.7382
Mean recovery (refined)	0.9054
Median recovery (model only)	0.7617
Median recovery (refined)	0.9652
Mean Δ recovery	+0.1672
Pairs where refinement helped	18/18
Mean time per pair (rasterize)	6.5 ms
Mean time per pair (model forward)	212.3 ms
Mean time per pair (refinement)	570.1 ms
Mean Shapely calls per pair	2205 (expected 2205)